

NVIDIA Inference Reference Architecture

A consolidated document compiled from the NVIDIA DSX Documentation

Source: <https://docs.nvidia.com/dsx/guides/inference-ra/>

Table of Contents

- 1. [Introduction](#)
 - 2. [Why Adopting This Architecture is Essential](#)
 - 3. [NVIDIA Open Models Across Modalities](#)
 - 4. [Common Component Combinations](#)
 - 5. [Component Layers](#)
 - 6. [Data Flow Diagrams](#)
 - 7. [Key Component Interactions](#)
 - 8. [Component Interaction Matrix](#)
 - 9. [Getting Started](#)
 - 10. [Example Workload: Large MoE LLM Inference](#)
 - 11. [Appendix](#)
-

1. Introduction

This document outlines a software architecture (called the NVIDIA Inference Reference Architecture) intended to help NVIDIA Cloud Partners (NCPs)—sometimes called operators—build a performant, cost-effective solution for large-scale AI inference workloads. It is intended to provide NCPs and ISVs with a Northstar definition that will best serve AI practitioners and cloud operators alike.

The NVIDIA Inference Reference Architecture is focused on providing a modern stack that meets the evolving needs of AI inference infrastructure. While AI workloads have traditionally been dominated by training, the rapid emergence of generative AI and reasoning models has shifted the paradigm toward inference-centric operations. The increasing complexity of serving these models in distributed environments has strengthened the need for a specialized, cloud-native inference infrastructure. Thus, this document will describe how to build such a solution that enables high-throughput, low-latency inference serving for next-generation AI applications.

Specifically, the NVIDIA Inference Reference Architecture provides an overview of a disaggregated, infrastructure-native architecture that is desired for world-class AI inference infrastructure. This term refers to a CSP-like (Cloud Service Provider) elastic cloud environment where compute, networking, and storage resources are allocated in an on-demand model optimized for inference workloads, rather than statically allocated or monolithically structured.

The NVIDIA Inference Reference Architecture provides a suite of complementary components that deliver high-performance, scalable AI inference. Each component is designed to work standalone or integrate seamlessly with others, allowing you to adopt the full stack or select specific components that address your requirements.

This architecture addresses the complete inference lifecycle:

- **Model Optimization:** Transform models for maximum GPU efficiency
 - **Inference Serving:** Handle requests for both traditional ML and GenAI workloads
 - **Memory Management:** Intelligent caching and high-speed data transfer across memory tiers
 - **Cloud Orchestration:** Kubernetes-native scaling, scheduling, and infrastructure management
 - **Performance Tooling:** Benchmarking, tuning, and configuration optimization
-

2. Why Adopting This Architecture is Essential

The shift to inference-centric AI, driven by generative models, necessitates a fundamental change in infrastructure. Traditional stacks, optimized for training, fail to meet the sustained, low-latency, and high-throughput demands of modern, large-scale inference. Adopting the NVIDIA Inference Reference Architecture is critical for:

- **Addressing the Generative AI Performance Gap:** Achieve the ultra-low latency and high throughput required by next-generation models like LLMs, delivering the industry-leading **Performance** detailed in the quantifiable metrics below.
- **Optimizing Cloud Economics:** The disaggregated, infrastructure-native approach enables truly elastic resource allocation, eliminating costly over-provisioning and improving capital efficiency for CSPs and NCPs, translating to a substantial improvement in **Cost & Efficiency (TCO)**.
- **Future-Proofing Your Inference Stack:** Provide a modern, cloud-native foundation that supports the complete inference lifecycle—from optimization to Kubernetes-native scaling, ensuring agility and long-term competitiveness in **Scalability & Agility (Time-to-Market)**.

Measuring the Value Proposition

The architecture's value is quantifiable, driving superior service delivery and business outcomes:

Performance:

- **Token/watt Improvement:** Achieve up to **50x increase in requests** served for low-latency workloads.
- **Compute Performance:** Benefit from **1.5x higher NVFP4 compute performance** and **2x faster attention processing** compared with previous generations.

Cost & Efficiency (TCO):

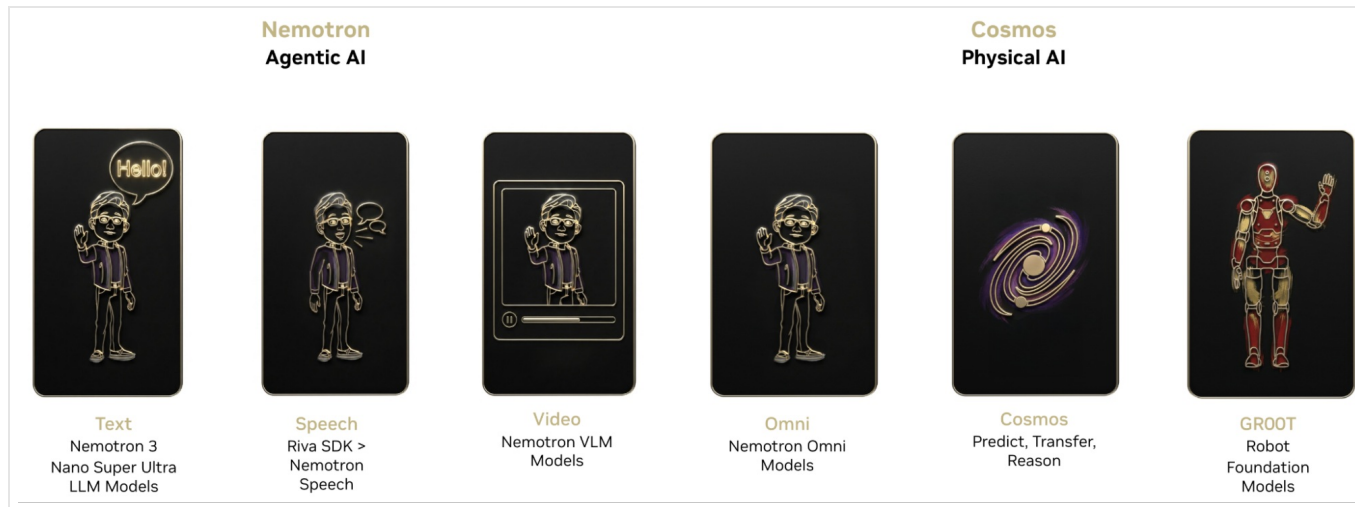
- **Total Cost of Ownership:** Realize up to **35x lower cost per million tokens** compared with previous-generation platforms.

Scalability & Agility (Time-to-Market):

- **Time-to-Deployment:** Accelerate system deployment and scaling from weeks to [BB] days/hours.
- **Autoscaling Efficiency:** Scale resources up or down by a factor of [CC]X within minutes.

By adopting this Northstar definition, NCPs and ISVs can deliver world-class, cost-effective, and highly performant AI services.

3. NVIDIA Open Models Across Modalities



AI requires trust and transparency, which is why NVIDIA is pioneering an open foundation for accelerated computing. This NVIDIA Inference Reference Architecture supports NVIDIA's open model family to empower developers to build specialized, intelligent agents. Designed for efficiency and real-world deployment, the architecture provides the foundation for accessible, high-performance AI solutions that can be customized for diverse industries.

- **Hardware co-design:** Building our own models ensures we continue to accelerate inference across model architectures
- **Software co-design:** All modalities are optimized to work on NVIDIA platforms and ready for scale
- **Data provenance:** Clear training and fine-tuning recipes with many datasets openly available

Modular by Design

Every component in this reference architecture can be:

- Adopted independently to solve specific challenges
- Combined incrementally as your needs evolve
- Integrated with existing infrastructure you already have in place

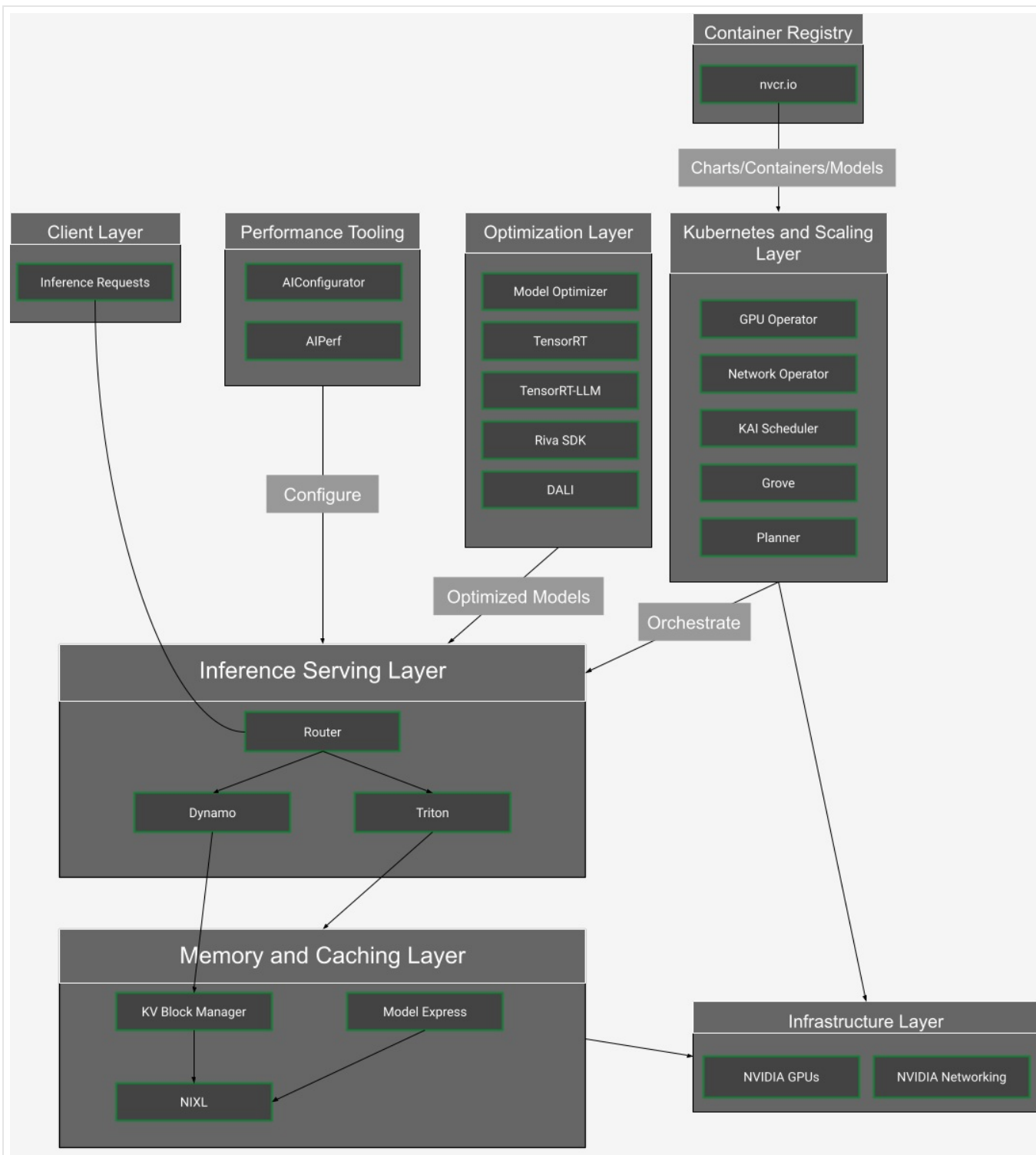
Whether you need just TensorRT for model runtime, NVIDIA Triton™ for serving, or a complete NVIDIA Dynamo stack for disaggregated LLM inference, the architecture scales to match your requirements at datacenter scale.

4. Common Component Combinations

These are typical adoption patterns showing how components complement each other:

Use Case	Core Components	Optional Additions
Basic ML Inference	TensorRT + Triton	DALI, GPU Operator
Speech/NLP Pipeline	Riva SDK + Triton	DALI, TensorRT
Single-Node LLM	TensorRT-LLM + Dynamo	Model Optimizer
Distributed LLM	Dynamo + KV Block Manager + NIXL + Router + TensorRT-LLM	Planner, Model Express, Model Optimizer
Kubernetes Deployment	GPU Operator + KAI Scheduler	Network Operator, Grove
Full GenAI Stack	Dynamo + NIXL + KV Block Manager + Router + Grove + KAI Scheduler + Planner	AIConfigurator, AIPerf

Architecture Overview



5. Component Layers

Each layer contains components that can be adopted independently or combined with others. Components within a layer often complement each other but do not require the full layer to function.

Infrastructure Layer

Dynamic Resource Allocation (DRA) enables flexible GPU sharing in Kubernetes clusters. DRA ComputeDomains are a construct for managing distributed shared memory (IMEX). Together with advanced topology-aware schedulers, DRA powers predictable multi-node disaggregated model inference at production (rack) scale.

Component	Purpose
NVIDIA IMEX	A facility that allows a set of GPUs to directly read / write each other's memory over a high-bandwidth NVLink. The connection may either be directly between GPUs on the same node or between GPUs on separate nodes connected by an NVSwitch.
NVIDIA NVLink	Scale-up networking fabric with high-bandwidth GPU-to-GPU communications for AI training, inference, and other demanding rack-scale GPU-accelerated workloads.
Dynamic Resource Allocation (DRA)	Controlled sharing and dynamic reconfiguration of GPUs. Enables ComputeDomains, which enable robust and secure Multi-Node NVLink (MNNVL) for NVIDIA GB200 and similar systems.

Optimization Layer

Components that prepare and optimize models for maximum inference performance. Use any combination based on your model types and requirements.

Component	Purpose
Model Optimizer	State-of-the-art optimization techniques including quantization, distillation, pruning, speculative decoding, and sparsity for more efficient model checkpoints to deploy on GPUs.
TensorRT	High-performance deep learning inference runtime for traditional Deep Learning models.
TensorRT-LLM	Purpose-built runtime with state-of-the-art inference optimization for GenAI models deployed on NVIDIA GPUs.

Deployment Layer

Cloud-native orchestration and infrastructure management. Start with GPU Operator for basic GPU management, then add components as scaling requirements grow. When deploying Dynamo with NVIDIA's Helm charts, you can optionally install Grove and KAI Scheduler in the same chart.

Component	Purpose
KAI Scheduler	Kubernetes scheduler optimized for GPU resource allocation in AI/ML workloads.
GPU Operator	Manages GPU nodes in Kubernetes clusters like CPU nodes.
Network Operator	Enables RDMA and GPUDirect networking for high-performance workloads.
Grove	K8s API for defining gang scheduling and topology awareness for multinode disaggregated serving.
Planner	Real-time performance tuning for prefill and decode given SLA constraints (TTFT and ITL).

Inference Serving Layer

The runtime engines that handle inference requests. Choose Triton for traditional ML workloads, Dynamo for GenAI, or both for mixed environments.

Component	Purpose
Dynamo	Purpose-built inference serving for GenAI workloads with disaggregated architecture support.
Router	Smart request routing leveraging KV cache hit rate and load balancing, with extensible algorithms for agentic workloads.
Gateway API Inference Extension	Integration of the k8s inference gateway with Dynamo.

Memory and Caching Layer

High-performance memory management and data transfer components. These unlock advanced capabilities like disaggregated serving and fast model loading when needed.

Component	Purpose
NIXL	Point-to-point data transfer between GPUs and G1-G4 memory/storage tiers.
KV Block Manager	Manages KV cache across memory tiers with customizable eviction policies.
Model Express	Fast model weight loading via caching and NIXL transfer, also leveraged for fault tolerance.

Performance Tooling

Tools for benchmarking and configuration. Use these independently or together to optimize your deployment.

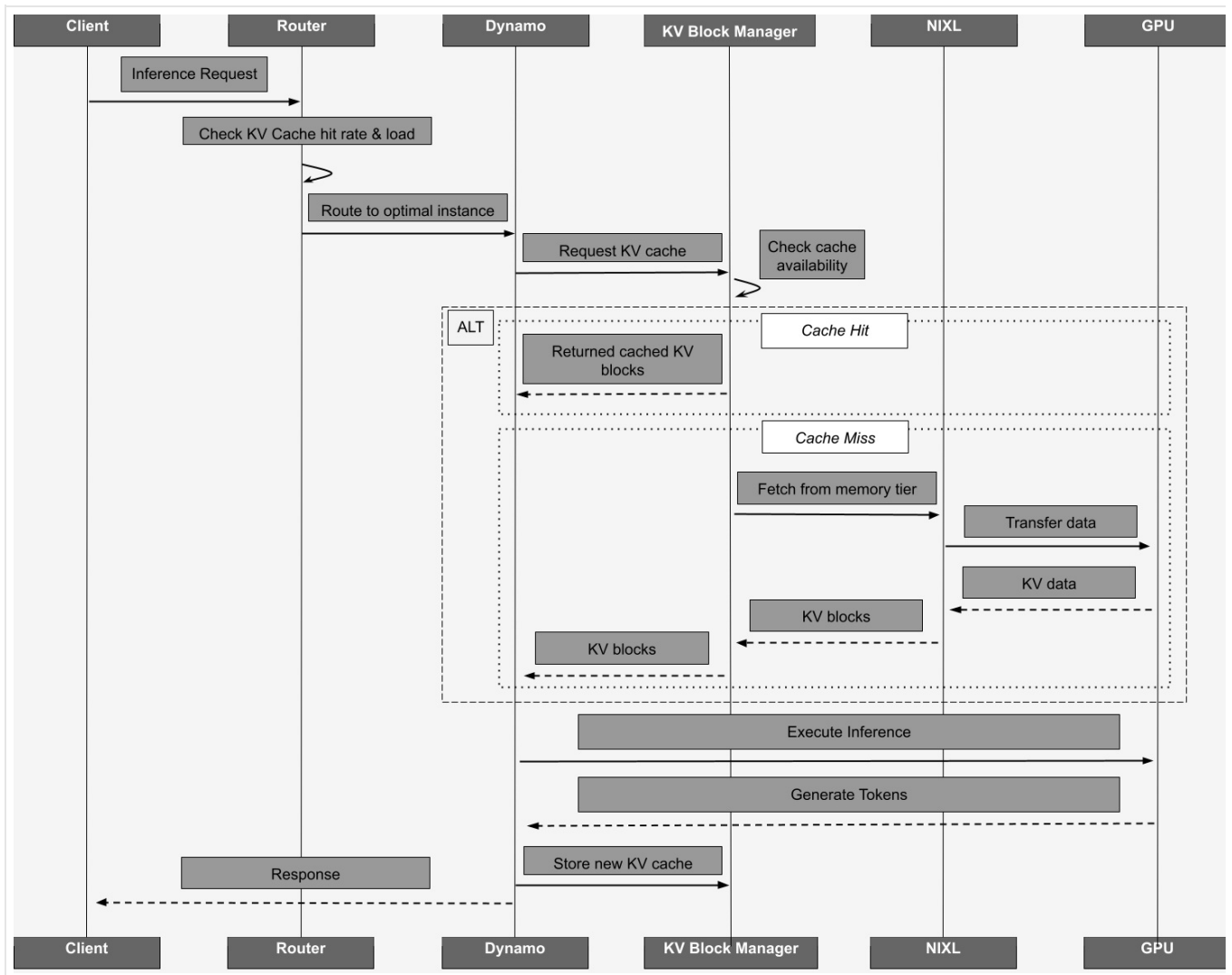
Component	Purpose
AIConfigurator	Performance estimation for aggregated vs disaggregated serving based on model, ISL/OSL, and hardware.
AIPerf	Distributed benchmarking tool (Python-based, highly extensible).

Container Registry

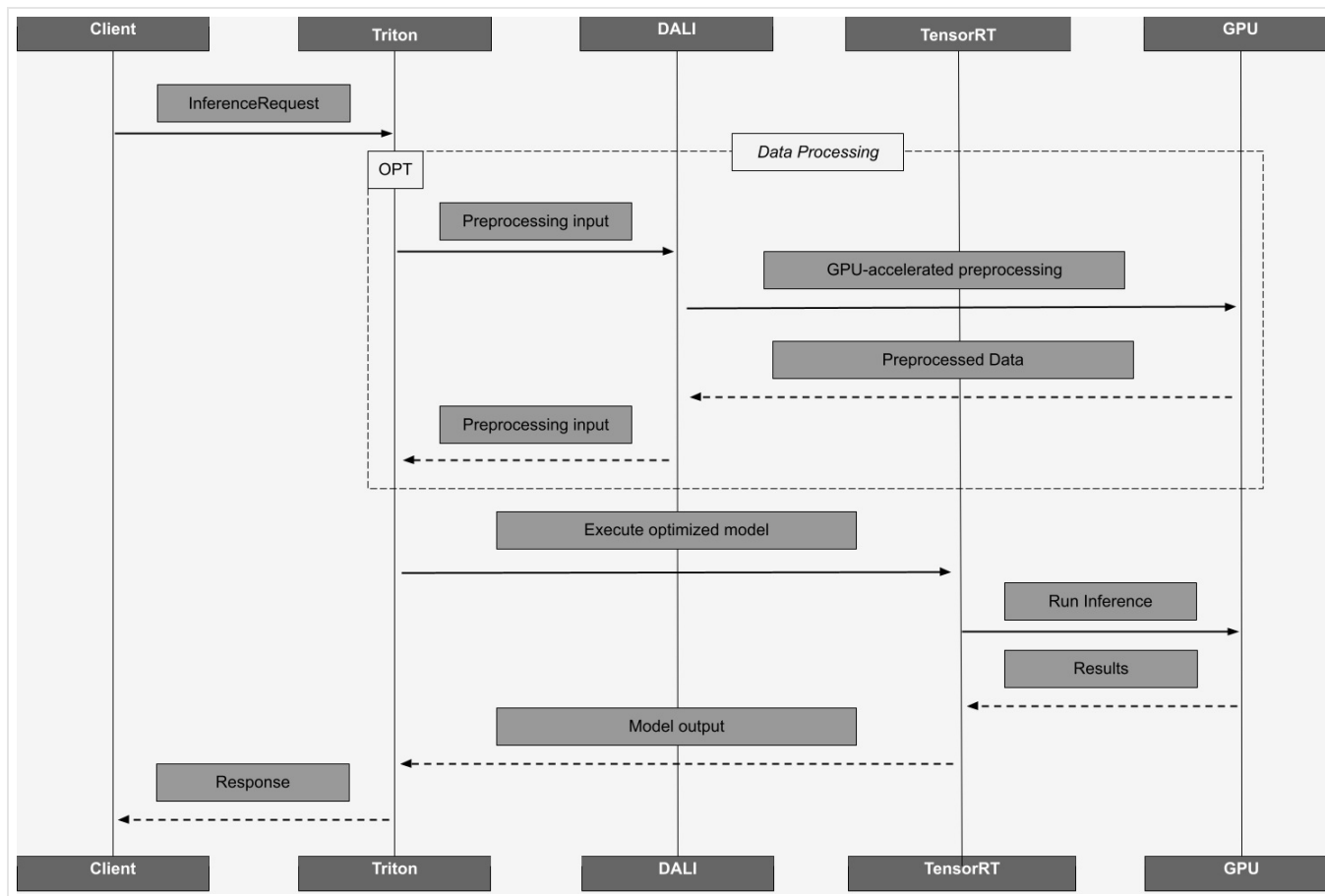
Component	Purpose
nvcr.io	GPU-optimized containers, pretrained models, SDKs, and Helm charts for cloud-to-edge deployment.

6. Data Flow Diagrams

GenAI/LLM Inference Flow

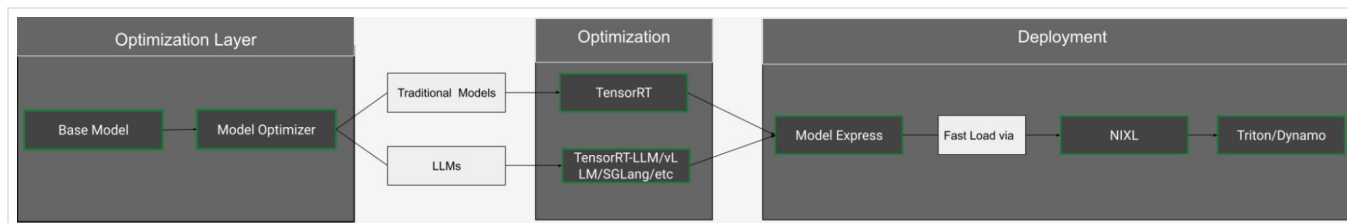


Traditional ML Inference Flow



Model Deployment Flow

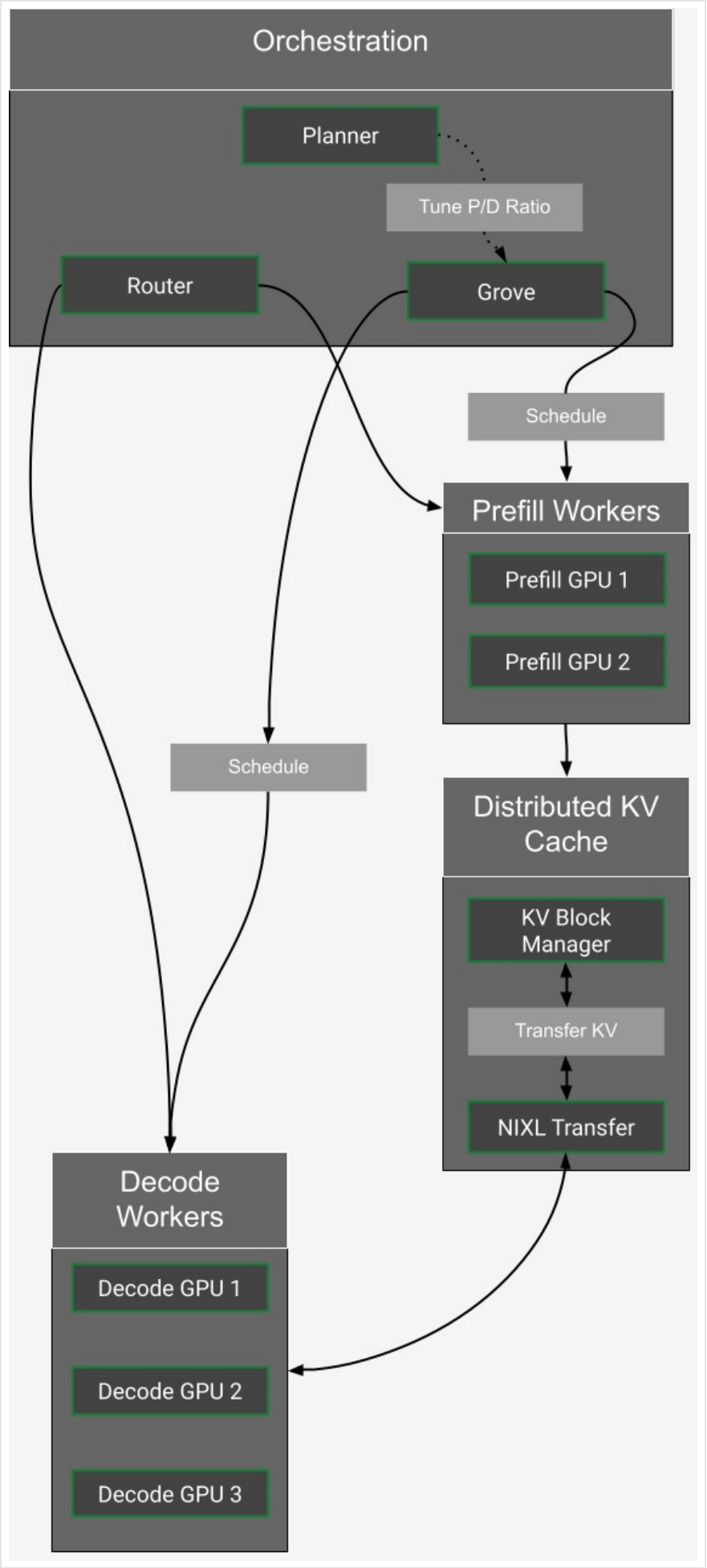
Components can be used independently or combined. This diagram shows a full pipeline, but you can enter at any stage.



7. Key Component Interactions

Disaggregated LLM Serving

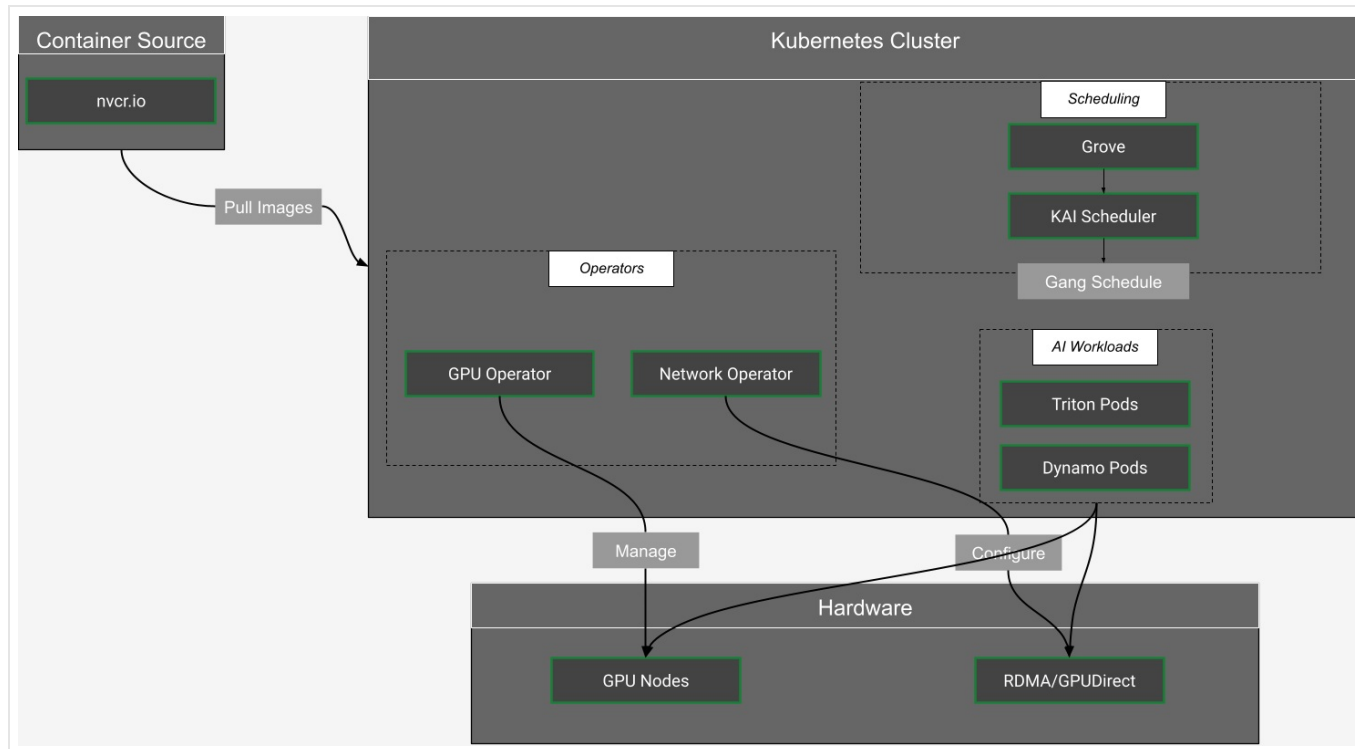
The architecture enables disaggregated prefill and decode phases for optimal LLM serving:



Key interactions:

- **Planner + Grove:** Planner determines optimal prefill/decode ratios based on SLA constraints; Grove handles gang scheduling.
- **Router + KV Block Manager:** Router uses cache hit rates and load information to make intelligent routing decisions.
- **KV Block Manager + NIXL:** Enables KV cache to span across memory tiers (G1-G4) with high-speed transfers.

Kubernetes Infrastructure Stack



Key interactions:

- **GPU Operator + Network Operator:** Together provide full infrastructure management for GPU clusters.
 - **KAI Scheduler + Grove:** KAI handles general GPU scheduling; Grove adds gang scheduling for multinode workloads.
 - **nvcv.io + Operators:** Pre-built containers simplify deployment of the entire stack.
-

8. Component Interaction Matrix

Component	Interacts With	Integration Type
Dynamo	KV Block Manager, Router, Planner, NIXL	Runtime
Router	Dynamo, Triton, KV Block Manager	Request routing
KV Block Manager	NIXL, Dynamo, Router	Memory management
NIXL	KV Block Manager, Model Express, GPUs	Data transfer
Model Express	NIXL, Triton, Dynamo	Model loading
Planner	Dynamo, Grove	Performance tuning
Grove	KAI Scheduler, Planner	Orchestration
KAI Scheduler	Grove, GPU Operator	Scheduling
GPU Operator	Network Operator, KAI Scheduler	Infrastructure
Network Operator	GPU Operator	Infrastructure
AIConfigurator	—	Planning
AIPerf	Triton, Dynamo	Benchmarking
Model Optimizer	TensorRT, TensorRT-LLM	Optimization
TensorRT-LLM	Dynamo, Model Optimizer	Optimization
nvcv.io	All components	Distribution

9. Getting Started

Choose the components that fit your needs. Here are common adoption paths:

Full Stack Deployment

1. **Infrastructure:** Deploy GPU Operator and Network Operator to your Kubernetes cluster.
2. **Containers:** Pull optimized containers from nvcr.io.
3. **Optimize:** Use Model Optimizer with TensorRT or TensorRT-LLM.
4. **Plan:** Use AIConfigurator to estimate performance and plan deployment topology.
5. **Deploy:** Use KAI Scheduler (add Grove for multinode) to deploy Triton or Dynamo.
6. **Tune:** Use AIPerf for benchmarking, Planner for runtime optimization.

Traditional ML Inference Only

1. **Optimize:** Use TensorRT to optimize your models.
2. **Serve:** Deploy with Triton Inference Server.
3. **Optional:** Add DALI for GPU-accelerated preprocessing.

GenAI/LLM Inference Only

1. **Optimize:** Use TensorRT-LLM to optimize your LLM.
2. **Serve:** Deploy with Dynamo.
3. **Scale:** Add KV Block Manager, NIXL, and Router for distributed inference.

Kubernetes Integration Only

1. **Deploy:** GPU Operator + Network Operator for infrastructure management.
 2. **Schedule:** KAI Scheduler for GPU-aware scheduling.
 3. **Scale:** Add Grove for gang scheduling if needed.
-

10. Example Workload: Large MoE LLM Inference

This implementation addressed multiple challenges often faced during the deployment of LLMs.

- **Difficult UX:** User experience is critical for distributed inference runtimes because managing large-scale inference systems is already complex, and poor usability further complicates matters. Developers need a clear, intuitive way to define, optimize, and update inference execution without wrestling with low-level infrastructure details. Without simple UX, inference runtimes remain inaccessible, prone to errors, and inefficient, hindering model deployment and innovation. A modern distributed inference stack must consider usability at its core—empowering developers to scale AI effortlessly for agentic workflows while ensuring correctness and performance.
- **GPU underutilization:** Traditional monolithic inference pipelines often leave GPUs idle due to the imbalance between prefill and decode stages. Prefill (which generates large prompt embeddings) is highly compute-intensive, while decode (which generates tokens) is latency-sensitive. A disaggregated approach that separates prefill and decode ensures optimal GPU utilization and increases overall throughput (DistServe).
- **Expensive KV cache re-computation:** When requests aren't efficiently routed, KV caches (intermediate states of the transformer model) often get flushed and recomputed, leading to wasted computation cycles and increased latency. KV-aware request routing eliminates redundant KV cache regeneration, significantly boosting efficiency. (DeepSeek)
- **Memory bottlenecks:** Large-scale inference workloads demand extensive KV cache storage, which can quickly overwhelm GPU memory capacity. KV cache offloading across memory hierarchies (HBM, DDR, NVMe or remote storage) enables models to scale beyond GPU memory limits and speeds up latency. (Mooncake, AIBrix, LMCache)
- **Fluctuating demand and inefficient GPU allocation:** Inference workloads are use-case specific and dynamic—demand surges inherently cause unpredictability, yet traditional serving stacks allocate GPUs statically. Dynamic GPU scheduling ensures that resources are allocated based on real-time demand, preventing over-provisioning and improving utilization (AzureTrace).
- **Inefficient data transfer:** Distributed inference workloads introduce unique and highly dynamic communication patterns that differ fundamentally from training. Unlike training, where worker roles remain largely static, inference requires real-time worker scaling, dynamic load balancing, and adaptive memory management—necessitating a communication layer that can efficiently handle these evolving requirements. Contemporary libraries are built for static, synchronous operations and lack the dynamicity needed for inference serving. While UCX provides high-performance networking, it requires deep networking expertise to configure correctly, making it impractical for broad inference use cases. Developers need a library optimized for inference workloads that can abstract heterogeneous memory (remote memory or storage) and dynamically select the best transport mechanism via a unified API.

To address the growing demands of distributed inference serving, NVIDIA built Dynamo. Dynamo tackles key challenges in scheduling, memory management, and data transfer. It employs KV-aware routing for optimized decoding, leveraging existing KV caches. For efficient global memory management at scale, it strategically stores and evicts KV caches across multiple memory tiers—GPU, CPU, SSD, and object storage—enhancing both time-to-first-token and overall throughput. Dynamo features NIXL (NVIDIA Inference tranXfer Library), a new data transfer engine designed for dynamic scaling and low-latency storage access.

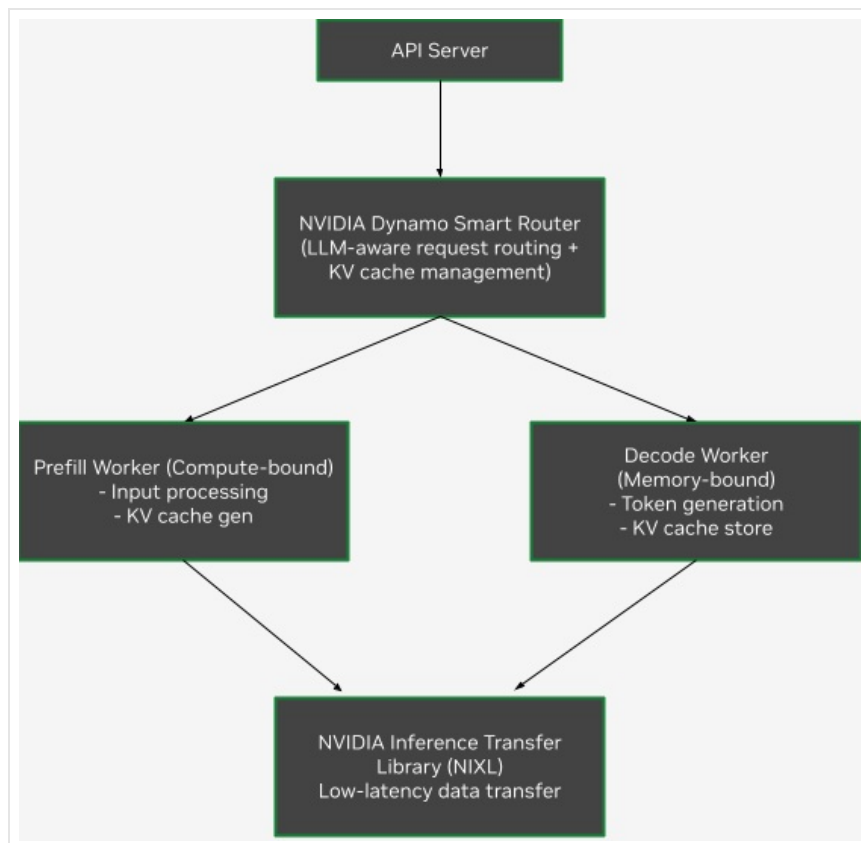
Architecture Overview (Dynamo)

Core Design Philosophy

NVIDIA Dynamo builds upon the success of NVIDIA Triton Inference Server™, introducing a modular architecture specifically optimized for distributed generative AI inference. The framework supports all major LLM frameworks including NVIDIA TensorRT-LLM, vLLM, and SGLang.

Key Architectural Components

1. Disaggregated Serving Architecture



2. NVIDIA Dynamo Planner

- Continuously monitors GPU capacity metrics in distributed inference environments
- Combines metrics with application SLOs (TTFT, inter-token latency)
- Makes real-time decisions on disaggregated vs aggregated serving
- Dynamic GPU worker allocation based on fluctuating demand

3. NVIDIA Dynamo Smart Router

- Tracks KV cache across large fleets of GPUs
- Hashes incoming requests and stores in Radix Tree
- Minimizes KV cache recomputation through overlap scoring
- Intelligent request routing considering cache hit rate and workload balance

4. NVIDIA Dynamo Distributed KV Cache Manager

- Hierarchical caching across GPU, node, and cluster levels
- KV cache offloading to cost-effective storage (CPU host memory, SSDs, networked storage)
- Intelligent eviction policies balancing lookup latency vs recomputation costs
- Framework-agnostic supporting PyTorch, SGLang, TensorRT-LLM, and vLLM

5. NVIDIA Inference Transfer Library (NIXL)

- High-throughput, low-latency point-to-point communication library
- Hardware-agnostic data movement API across memory hierarchies
- Supports heterogeneous data paths (NVLink, InfiniBand, RoCE, Ethernet)
- Optimized for inference data movement with nonblocking, noncontiguous transfers

Performance Characteristics

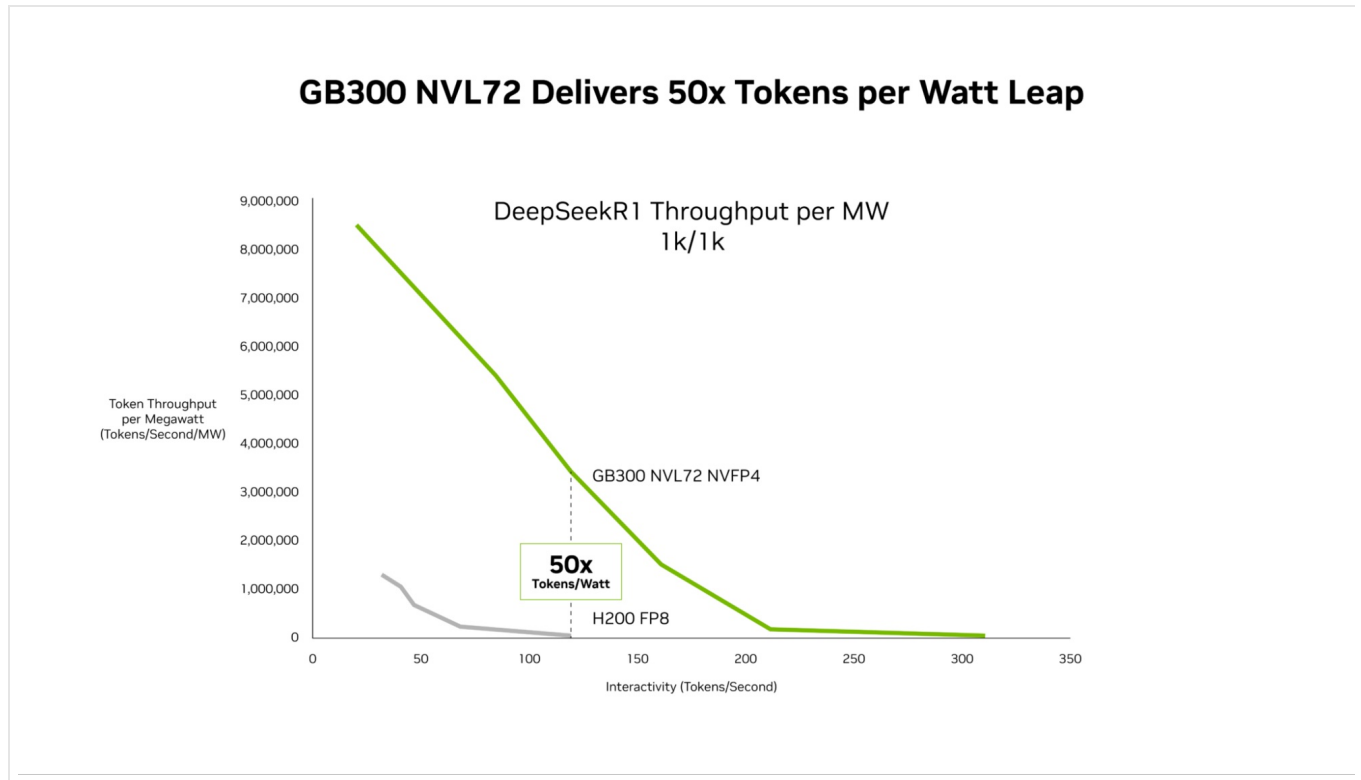
DeepSeek-R1 Model Performance

Deployment Environment: NVIDIA GB300 NVL72

- **Model Size:** DeepSeek-R1 671B
- **Configuration:** Disaggregated serving with EP4DP16 context, EP64DP3 generation

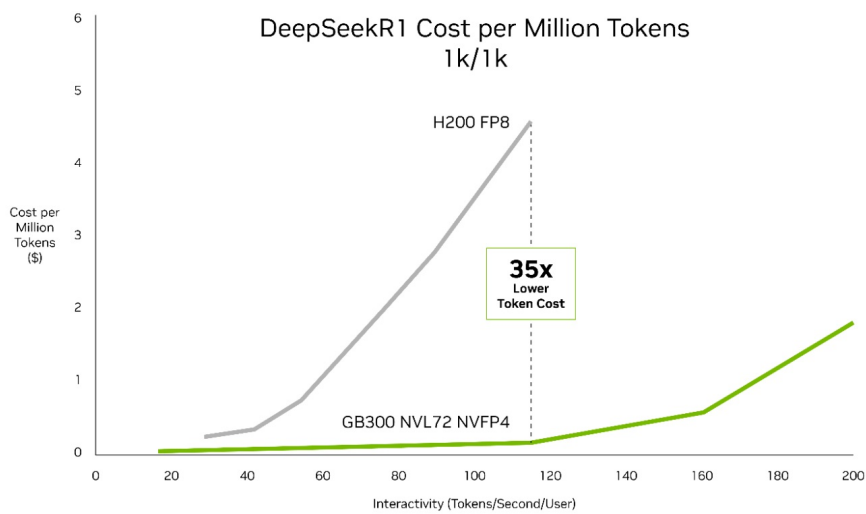
General Performance Gains

Token/watt Improvement: 50x increase in requests served.



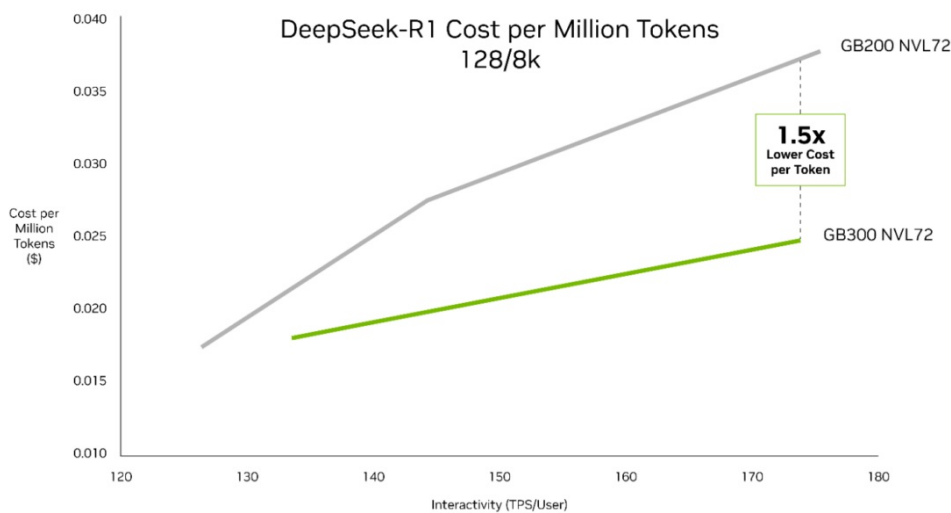
Improved TCO: 35x lower cost per million tokens compared with the Hopper platform.

GB300 NVL72 Delivers 35x Reduction in Token Cost



Compute Performance: 1.5x higher NVFP4 compute performance and 2x faster attention processing compared with the GB200 platform.

GB300 NVL72 Delivers Large Leap for Long Context AI DS-R1, NVFP4, 128k/8k



Technical Implementation Details

Model Optimization

The model was quantized to NVFP4 with NVIDIA Model Optimizer v0.23.0.

Datasets:

--	--	--	--

Dataset	Purpose	Data collection method	Labeling method
cnn_dailymail	Calibration Dataset	Automated	Unknown
MMLU	Evaluation Dataset	Unknown	N/A

Prefill vs Decode Separation

Prefill Phase:

- Compute-bound operation
- Processes user input to generate first output token
- Optimized with low tensor parallelism to reduce communication overhead

Decode Phase:

- Memory-bound operation
- Generates subsequent tokens
- Optimized with high tensor parallelism for improved memory operations

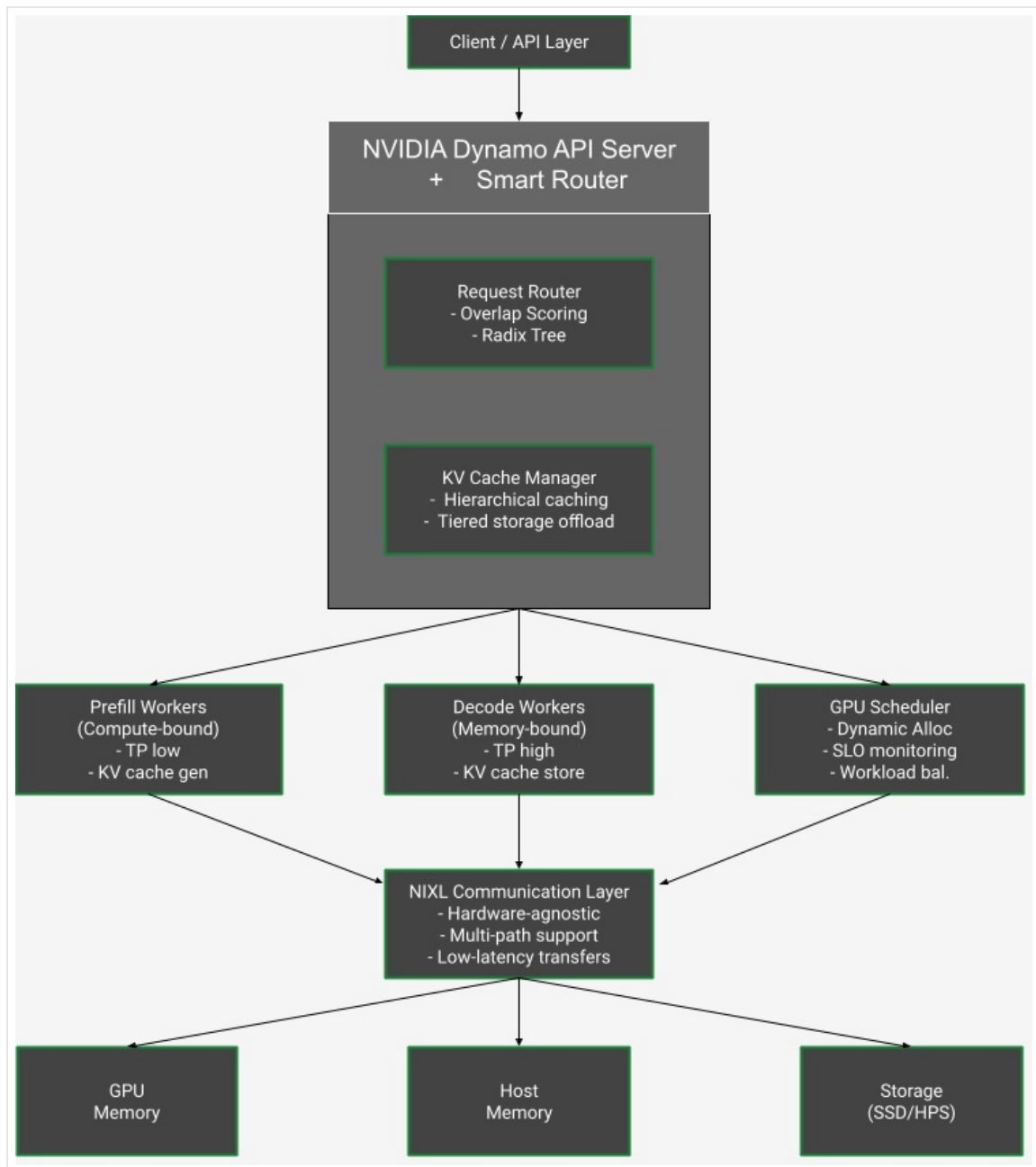
Cost Optimization Strategies

- **Disaggregated Serving:** Independent optimization of each phase
- **KV Cache Management:** Tiered storage hierarchy reducing GPU memory requirements
- **Dynamic Scheduling:** Resource allocation based on real-time workload demands
- **Request Routing:** Minimizing computational costs through cache reuse

Integration Capabilities

- **Framework Support:** PyTorch, SGLang, NVIDIA TensorRT-LLM, vLLM
- **Networking:** NVLink, InfiniBand (Quantum), Ethernet (Spectrum)
- **Storage:** GPUDirect Storage, S3 integration
- **Deployment:** Kubernetes compatible, available via NVIDIA NIM microservices

Reference Architecture



Deployment Recommendations

- **Scale-Out Strategy:** Start with disaggregated serving for high-throughput workloads
 - **Memory Hierarchy:** Implement tiered KV cache management for cost optimization
 - **Dynamic Allocation:** Utilize Dynamo Planner for real-time resource optimization
 - **Framework Integration:** Leverage existing TensorRT-LLM or vLLM deployments
 - **Network Optimization:** Configure NVLink for intra-node, InfiniBand for inter-node communication
-

11. Appendix

Component	Interacts With	Integration Type
Riva SDK	Speech recognition, synthesis, and NLP inference APIs	Speech SDK
DALI	GPU-accelerated data loading and preprocessing for deep learning pipelines	Data Processing
Triton Inference Server	Multi-framework model serving supporting TensorRT, PyTorch, ONNX, OpenVINO, Python, RAPIDS FIL, and more	Inference Serving Software

